

BRIDGE

NightHack Build Packet

The complete six-hour plan for a production-ready API migration demo

THE DEMO PROMISE

A breaking API change becomes a repo-specific, tested draft pull request inside a live collaborative migration room.

Friday, July 24, 2026 • 6:30 p.m.-12:30 a.m. PDT

Production URL • Live product demo • No slide deck

Executive Snapshot

THE ONLY BUILD

One provider. One breaking change. One TypeScript repository. One deterministic migration. One draft PR. One verified CI pass. One shared room.

Must prove	Do not build	Final demo state
Detect → impact → patch → validate → coordinate	Generic agent, broad provider support, billing, auto-merge	Ready for review with real PR and exact-SHA green CI

Document map

1. Bridge NightHack Build Packet — 00_READ_ME_FIRST.md
2. Product and Demo Brief — 01_PRODUCT_AND_DEMO_BRIEF.md
3. NightHack Execution Plan — 02_NIGHTHACK_EXECUTION_PLAN.md
4. Bridge NightHack MVP PRD — 03_MVP_PRD.md
5. Technical Architecture — 04_TECHNICAL_ARCHITECTURE.md
6. Data Model and API Contracts — 05_DATA_MODEL_AND_API_CONTRACTS.md
7. Demo Fixture Specification — 06_DEMO_FIXTURE_SPEC.md
8. UI/UX and Design System — 07_UI_UX_AND_DESIGN_SYSTEM.md
9. Landing Page Copy and Design Specification — 08_LANDING_PAGE_COPY_AND_SPEC.md
10. Demo Script and Runbook — 09_DEMO_SCRIPT_AND_RUNBOOK.md
11. Pitch and Judge Q&A — 10_PITCH_AND_JUDGE_QA.md
12. QA Test Plan — 11_QA_TEST_PLAN.md
13. Deployment, Security, and Environment Checklist — 12_DEPLOYMENT_SECURITY_AND_ENV.md
14. Risk Register and Fallbacks — 13_RISK_REGISTER_AND_FALLBACKS.md
15. Team Roles and War Room — 14_TEAM_ROLES_AND_WAR_ROOM.md
16. Post-Hackathon Roadmap — 15_POST_HACKATHON_ROADMAP.md
17. Claude Code Master Prompt — 16_CLAUDE_CODE_MASTER_PROMPT.md
18. Final Submission Checklist — 17_FINAL_SUBMISSION_CHECKLIST.md

Bridge NightHack Build Packet

Event: Founders Inc NightHack

Build window: Friday, July 24, 2026, 6:30 p.m.-12:30 a.m. PDT

Constraint: Production deployment and live product demo; no localhost and no slide deck.

The only goal

Ship one undeniable vertical slice:

A third-party API introduces a breaking change. Bridge detects it, identifies the exact customer code that will break, creates a bounded migration patch, validates it in CI, opens a draft pull request, and gives the provider and customer a shared live migration room.

Do not build a general coding agent, an API documentation platform, an SDK generator, a broad observability product, or a generic collaboration tool. The product is the end-to-end migration event.

What “demo-ready” means

A judge can watch the full story in four to six minutes without hearing architecture excuses:

19. A provider change exists as two hosted OpenAPI files.
20. Bridge detects and summarizes the breaking change.
21. Bridge finds two or three impacted files in a real GitHub repository.
22. Bridge creates a migration run with a visible timeline.
23. Bridge creates a branch, commits a deterministic patch, and opens a draft PR.
24. GitHub Actions returns a real passing check.
25. Two browser sessions appear in the same migration room with presence, comments, and an approval action.
26. The app is deployed on a public URL and remains usable after a refresh.

Non-negotiable priorities

P0: deployed app, one real GitHub repo, one real PR, one real passing CI run, coherent demo story.

P1: impacted-file evidence, migration timeline, multiplayer presence, approval.

P2: polished animations, multiple providers, generalized AI patching, billing, onboarding.

Packet map

- `01\_PRODUCT\_AND\_DEMO\_BRIEF.md` - product thesis and winning demo story.
- `02\_NIGHTHACK\_EXECUTION\_PLAN.md` - six-hour schedule, gates, and cut rules.
- `03\_MVP\_PRD.md` - exact NightHack product requirements and acceptance criteria.
- `04\_TECHNICAL\_ARCHITECTURE.md` - system design and critical implementation choices.
- `05\_DATA\_MODEL\_AND\_API\_CONTRACTS.md` - database tables, states, events, and endpoints.
- `06\_DEMO\_FIXTURE\_SPEC.md` - controlled provider change and sample customer repository.
- `07\_UI\_UX\_AND\_DESIGN\_SYSTEM.md` - screens, component states, visual system, and responsive behavior.
- `08\_LANDING\_PAGE\_COPY\_AND\_SPEC.md` - production landing-page copy and layout.
- `09\_DEMO\_SCRIPT\_AND\_RUNBOOK.md` - exact live demo sequence, narration, and resets.
- `10\_PITCH\_AND\_JUDGE\_QA.md` - spoken pitch, objections, and concise answers.

BRIDGE / NIGHTHACK BUILD PACKET

- `11\_QA\_TEST\_PLAN.md` - critical-path tests, failure modes, and release gate.
- `12\_DEPLOYMENT\_SECURITY\_AND\_ENV.md` - deployment checklist, permissions, secrets, and observability.
- `13\_RISK\_REGISTER\_AND\_FALLBACKS.md` - failure triggers and fallback modes.
- `14\_TEAM\_ROLES\_AND\_WAR\_ROOM.md` - two-person and three-person operating plans.
- `15\_POST\_HACKATHON\_ROADMAP.md` - 72-hour, 30-day, and 90-day path.
- `16\_CLAUDE\_CODE\_MASTER\_PROMPT.md` - build-agent instructions for disciplined execution.
- `17\_FINAL\_SUBMISSION\_CHECKLIST.md` - final hour and demo-table checklist.

Rules-safe preparation before Friday

Prepare research, architecture, accounts, credentials, design references, copy, demo scripts, and checklists. Confirm the organizer's rules before writing judged product functionality. If prebuilding implementation is prohibited, do not hide or disguise earlier work; begin product code when the clock starts. You can still remove avoidable setup risk by verifying account access, billing, domain ownership, GitHub organization permissions, and deployment credentials without implementing Bridge.

Product and Demo Brief

Product definition

Bridge is **migration orchestration infrastructure for external API changes**. It converts a provider-side breaking change into repo-specific, tested migration work for the customer.

The core loop is:

Detect -> Map impact -> Plan -> Patch -> Validate -> Coordinate -> Review

Problem

External APIs change faster than engineering teams can reliably monitor, interpret, map into their own code, patch, test, and coordinate. Existing tools solve fragments: changelogs announce changes, API diff tools compare schemas, coding agents modify repositories, and CI validates commits. Nobody owns the entire migration event.

Target users

Primary NightHack user

A platform or application engineer maintaining a production TypeScript integration in GitHub.

Long-term buyers

- Engineering teams with several business-critical third-party APIs.
- Platform engineering and developer productivity teams.
- API providers that want customers to migrate faster and create fewer support tickets.
- Systems integrators managing many customer integrations.

Sharp positioning

Bridge turns external API changes into tested pull requests.

Alternative one-liners:

- ****Dependabot for APIs, with migration execution.****
- ****The control plane for API migrations.****
- ****From breaking change to reviewed PR.****

What Bridge is not

- Not a general AI coding agent.
- Not an API client or API testing suite.
- Not an SDK generator.
- Not a documentation portal.
- Not a dependency bot for package versions.
- Not a Slack replacement.
- Not autonomous code merging.

NightHack proof

The demo must prove four claims:

27. ****Detection:**** Bridge understands a real provider contract change.
28. ****Relevance:**** Bridge maps that change to exact customer code paths.

29. ****Execution:**** Bridge makes a safe patch and validates it with real CI.
30. ****Coordination:**** Bridge gives provider and customer a shared, auditable room for the migration.

Controlled demo story

Use a fictional provider called **AtlasPay** so the team controls every artifact and failure mode. AtlasPay v2 removes [payment_method](#) and requires [payment_method_id](#) in [POST /payments](#). A customer TypeScript repository still uses the old field in three places.

Bridge shows the OpenAPI diff, finds the three call sites, creates a migration plan, performs a deterministic key rename, triggers GitHub Actions, opens a draft PR, and updates the room to [ready_for_review](#) after CI passes.

Winning visual moment

The strongest moment is not the spec diff. It is the transition from a red migration run to a green draft PR while a second browser participant is visibly present and approves the proposed migration. That compresses detection, execution, proof, and multiplayer collaboration into one screen.

Success criteria

- The entire story works from a deployed URL.
- The pull request is real and links back to the migration room.
- CI evidence is real, not a fabricated green badge.
- The patch is small enough to inspect in seconds.
- A judge understands the category without a slide.
- The demo completes in under six minutes.

NightHack Execution Plan

Operating principle

Six hours is not enough to build the company. It is enough to build one credible proof. Every task must improve the live demo path. Cut anything that does not.

Before the clock starts

Rules-safe preparation:

- Confirm organizer rules and what counts as prebuilt work.
- Ensure every teammate can access GitHub, deployment, Supabase, model provider, and domain accounts.
- Write down environment-variable names without storing secrets in documents.
- Decide the sample migration and exact before/after code.
- Prepare the demo narration and fallback hierarchy.
- Charge laptops, bring adapters, test hotspot access, and disable disruptive OS updates.
- Create an incident contact thread for the team.

Do not prebuild judged functionality if the event rules prohibit it.

Six-hour schedule

6:30-6:40 - Lock scope and create the board

- Read the single demo sentence aloud.
- Assign owners.
- Create issues for P0 tasks only.
- Set a visible cut list.
- Start a production deployment immediately, even if it only shows a status page.

Gate: public URL loads for every teammate.

6:40-7:15 - Foundation and data model

- Next.js app and production environment.
- Supabase tables and seed migration run.
- GitHub App authentication or a narrowly scoped fallback token owned by the backend.
- Basic migration-room route with server-fetched data.

Gate: a seeded run survives refresh on the public URL.

7:15-8:00 - Provider change ingestion

- Host `atlaspay-v1.yaml` and `atlaspay-v2.yaml`.
- Run deterministic OpenAPI diff or a controlled structured diff.
- Normalize the result into one `ProviderChange` record.
- Render the change summary and severity.

Gate: clicking “Analyze change” creates a real run and displays the removed/required field change.

8:00-8:45 - Impact scan and migration plan

- Fetch the sample repo tree or selected files through GitHub.
- Search TypeScript source for the deprecated field.
- Save impacted file paths, line evidence, and confidence.
- Generate a bounded migration plan.

Gate: the room shows the correct three impacted files with snippets.

8:45-9:35 - Patch, branch, commit, and draft PR

- Create a migration branch.
- Apply the deterministic key rename.
- Commit only the intended files.
- Open a draft pull request with Bridge summary, evidence, and room link.

Gate: the real PR exists and the diff is correct.

9:35-10:15 - Validation and checks

- Trigger or observe GitHub Actions on the branch.
- Poll workflow/check state from the backend.
- Store run URL, status, conclusion, and timestamps.
- Transition the migration state only after verified success.

Gate: the room displays a real GitHub check link and verified passing conclusion.

10:15-10:50 - Multiplayer migration room

- Add Supabase Realtime presence.
- Show participant avatars and roles.
- Add comments and one approval action.
- Broadcast run-status updates to both sessions.

Gate: two browsers see each other and the approval/status update without refreshing.

10:50-11:20 - Demo-quality interface

- Tighten layout hierarchy and empty/loading/error states.
- Add progress animation only where state is real.
- Make links to GitHub obvious.
- Add a compact “why this changed” explanation.

Gate: a new viewer can explain the product after 30 seconds.

11:20-11:45 - Integration freeze

- Stop adding features.
- Run the complete reset-and-demo flow twice.
- Fix only P0 and demo-blocking P1 issues.
- Capture backup screenshots and a short screen recording.

Gate: two consecutive successful rehearsals.

11:45-12:15 - Rehearsal and contingency validation

- One presenter speaks; one operator controls the second browser; one engineer watches logs.
- Rehearse the four-to-six-minute version three times.
- Verify backup migration run, backup PR, and backup CI evidence.
- Confirm all tabs, zoom levels, and credentials.

Gate: presenter can recover from any single failed integration in under 15 seconds.

12:15-12:30 - Final buffer

- No new commits unless the current build is broken.
- Pin the production deployment.
- Open all demo tabs.
- Clear noisy notifications.
- Breathe and run the product story, not an architecture lecture.

Cut order

Cut in this order when behind:

31. AI-generated prose and explanations.
32. Provider-side admin console.
33. Authentication polish.
34. Generalized repository installation flow.
35. Multiple API changes.
36. Fancy animations.
37. Comments.
38. Approval UI.

Never cut:

- public deployment,
- correct impacted files,
- real PR,
- real passing CI evidence,
- coherent migration-room state,
- demo reset capability.

Decision rules

- If a task exceeds 25 minutes without producing a demo-visible result, simplify or replace it.
- Use deterministic code for the migration recipe; do not put LLM output on the critical path.
- Keep external integrations behind adapters so a fallback implementation can preserve the UI flow.
- Never fabricate a successful external action. A cached successful run is acceptable only when clearly presented as a completed prior run from the same product.

Bridge NightHack MVP PRD

Goal

Demonstrate that Bridge can transform one external API breaking change into a safe, verified, collaborative migration for one GitHub repository.

Primary user story

As an engineer maintaining an AtlasPay integration, I want Bridge to identify code affected by the AtlasPay v2 contract change, propose and apply a bounded migration, validate it, and open a draft PR so I can review the change before production breaks.

Required product surfaces

1. Change intake

- Display provider name, version change, source URLs, severity, and detected breaking operations.
- Button: `Analyze change`.
- Prevent duplicate runs for the same repo and change unless the user explicitly retries.

2. Migration room

- Run title and status.
- Provider change summary.
- Impacted files with snippets and reason.
- Migration plan.
- Activity timeline.
- Participants/presence.
- Comment composer.
- Approval action.
- PR and CI evidence panel.

3. GitHub output

- Migration branch.
- Commit with deterministic patch.
- Draft PR.
- PR body includes change summary, impacted files, validation result, and Bridge room URL.

Functional requirements

FR-1 Change normalization

Given the two AtlasPay OpenAPI specs, Bridge records:

- removed field: `payment\_method`,
- added required field: `payment\_method\_id`,
- affected operation: `POST /payments`,
- risk: breaking,
- recommended migration: rename the request property.

FR-2 Impact discovery

Bridge searches selected TypeScript files and returns every syntactic object key named `payment_method` within the target integration scope. Each match stores file path, line number, snippet, and confidence.

FR-3 Patch generation

Bridge changes only the targeted object keys to [payment_method_id](#). It must not replace string values, comments, documentation text, unrelated variable names, or files outside the selected scope.

FR-4 GitHub branch and PR

Bridge creates a branch named like [bridge/atlaspay-v2-<short-id>](#), commits the patch, and opens a draft PR against the repository's default branch.

FR-5 Validation

Bridge observes a real GitHub Actions/check result for the migration commit. The migration cannot reach [ready_for_review](#) until the latest commit has a verified successful conclusion.

FR-6 Collaboration

At least two browser clients can join the same room, see presence, post comments, and observe status updates without refreshing.

FR-7 Approval

An authenticated or demo-role participant can approve the migration plan. Approval is recorded with participant, timestamp, and current plan version.

FR-8 Auditability

Every state transition creates an append-only activity event with actor, type, timestamp, and evidence link when relevant.

State machine

[queued](#) -> [analyzing_change](#) -> [scanning_repo](#) -> [planning](#) -> [patching](#) -> [validating](#) -> [ready_for_review](#)

Failure states:

[analysis_failed](#), [scan_failed](#), [patch_failed](#), [validation_failed](#), [cancelled](#)

Retry returns to the last safe stage and increments [attempt](#).

Non-functional requirements

- Public HTTPS deployment.
- Refresh-safe data.
- No secret in browser bundles.
- Backend-only GitHub credentials.
- P95 room load under three seconds for the demo dataset.
- Clear loading, empty, success, and error states.
- Idempotent branch/PR creation for a run.
- Logs include run ID but never tokens or source-file contents beyond controlled snippets.

Acceptance criteria

The MVP is accepted only when:

- A run starts from the deployed UI.
- It finds exactly the expected impacted files.
- The PR diff contains only intended changes.
- The PR is a draft.
- The app links to a real passing GitHub run/check.
- Two sessions show live presence.

- A comment and approval replicate live.
- A full rehearsal succeeds twice from reset.

Explicit non-goals

- Supporting arbitrary providers or languages.
- Automatically merging.
- Installing across arbitrary customer organizations during the demo.
- Full semantic call-graph analysis.
- Self-hosted runners.
- Billing.
- Enterprise SSO.
- Provider broadcast campaigns.
- Autonomous remediation without human review.

Technical Architecture

Architecture principle

Deterministic first, agent second. The critical migration path must be inspectable and reproducible. An LLM may summarize the change or explain evidence, but it must not be the only mechanism responsible for the patch.

Recommended NightHack stack

- ****Frontend/server:**** Next.js with TypeScript.
- ****Deployment:**** Vercel or another production platform already understood by the team.
- ****Database/auth/realtime:**** Supabase Postgres, Auth or demo identities, and Realtime Presence/Broadcast.
- ****Source control integration:**** GitHub App using installation tokens; use the minimum repository permissions required.
- ****Change detector:**** `oasdiff`, `openapi-diff`, or a controlled parser for the single AtlasPay change.
- ****Impact scanner:**** TypeScript AST where fast; otherwise a constrained structural search with exact key validation.
- ****Patch engine:**** deterministic AST transform or narrowly guarded text transform.
- ****Validation:**** GitHub Actions and check/workflow polling.
- ****Optional AI:**** one server-side model call for human-readable migration explanation, never required for success.

System components

Web application

Responsibilities:

- Render change intake and migration room.
- Subscribe to room presence and status updates.
- Send commands to server routes.
- Never receive GitHub private keys or installation tokens.

Orchestrator

A server-side stage runner that:

39. acquires a run lock,
40. verifies current state,
41. executes one stage,
42. writes outputs and an event,
43. transitions state,
44. broadcasts the update.

Do not build a complex queue. A database-backed stage lock plus idempotent routes is enough for the demo.

Provider change adapter

Input:

- old spec URL,
- new spec URL,
- provider metadata.

Output:

- normalized operations,
- breaking-change records,
- migration hints.

Repository adapter

Responsibilities:

- obtain installation token,
- fetch repo metadata and file contents,
- create branch,
- write blobs/commits or update files,
- create draft PR,
- fetch check/workflow status.

Impact scanner

For the AtlasPay demo, scope the scan to `src/**/*.ts` and `src/**/*.tsx`. Verify each match is an object-property key in a call that belongs to the AtlasPay integration. Save evidence snippets before patching.

Patch engine

Preferred:

- parse TypeScript,
- find targeted property assignments,
- rename the key,
- print the modified source,
- preserve unrelated formatting as much as practical.

Fallback:

- exact replacement only inside preselected files and only when a guard proves the line contains the AtlasPay request object.

Validation adapter

- Capture migration commit SHA.
- Trigger or observe workflow execution.
- Poll status with capped exponential backoff.
- Accept success only for the migration commit SHA.
- Store external run URL and conclusion.

Realtime room

Use Presence for participants and Broadcast or database changes for comments/status. Persist comments and approvals in Postgres; presence can remain ephemeral.

GitHub App permissions

Start from least privilege. Expected repository permissions:

- Metadata: read.
- Contents: read/write.
- Pull requests: write.
- Checks: read, or write only if Bridge creates its own check run.
- Actions: read if polling workflow runs.
- Workflows: avoid write unless Bridge truly edits workflow files.

Subscribe only to events needed for the demo, such as installation, push, pull request, check run, check suite, or workflow run depending on implementation.

Request flow

45. Browser posts `start migration`.
46. Server creates run and returns run ID.
47. Stage runner analyzes the specs.
48. Stage runner scans selected repository files.
49. Stage runner saves plan and waits for approval, or auto-approves in demo mode with a visible actor.
50. Stage runner creates branch and commit.
51. Stage runner opens draft PR.
52. Validation adapter tracks CI for exact SHA.
53. On success, run becomes `ready\_for\_review`.
54. Every stage persists an event and broadcasts an update.

Reliability requirements

- Unique constraint on `(provider\_change\_id, repository\_id, attempt)`.
- Idempotency key on branch and PR creation.
- Run lock with expiry to prevent duplicate workers.
- Store external IDs immediately after creation.
- Never retry a write blindly after an unknown network result; first query GitHub by branch/PR marker.
- Limit file size and total scanned bytes.
- Sanitize snippets displayed in UI.

Suggested repository structure

```
app/  
  change/[changeId]/page.tsx  
  room/[runId]/page.tsx  
  api/runs/start/route.ts  
  api/runs/[runId]/advance/route.ts  
  api/github/webhook/route.ts  
components/  
  change-summary.tsx  
  migration-timeline.tsx  
  impacted-files.tsx  
  evidence-panel.tsx  
  presence-stack.tsx  
lib/  
  github/  
  openapi/  
  scanner/  
  patcher/  
  validation/  
  realtime/  
  db/  
  state-machine/  
supabase/migrations/  
fixtures/
```

Production-vs-demo seams

Define interfaces for [ChangeDetector](#), [RepositoryClient](#), [PatchEngine](#), and [ValidationClient](#). The demo can use one implementation per interface without spreading AtlasPay-specific logic across the UI. This makes fallbacks possible and prevents the hack from becoming unmaintainable.

Data Model and API Contracts

Core tables

providers

- `id uuid primary key`
- `slug text unique`
- `name text`
- `logo\_url text nullable`
- `created\_at timestamptz`

provider_changes

- `id uuid primary key`
- `provider\_id uuid`
- `from\_version text`
- `to\_version text`
- `old\_spec\_url text`
- `new\_spec\_url text`
- `summary text`
- `severity text`
- `normalized\_diff jsonb`
- `created\_at timestamptz`

repositories

- `id uuid primary key`
- `github\_installation\_id bigint`
- `owner text`
- `name text`
- `default\_branch text`
- `created\_at timestamptz`

migration_runs

- `id uuid primary key`
- `provider\_change\_id uuid`
- `repository\_id uuid`
- `status text`
- `attempt int default 1`
- `current\_stage text`
- `plan\_version int default 1`
- `branch\_name text nullable`
- `commit\_sha text nullable`
- `pull\_request\_number int nullable`
- `pull\_request\_url text nullable`
- `validation\_url text nullable`
- `validation\_status text nullable`
- `validation\_conclusion text nullable`
- `error\_code text nullable`
- `error\_message text nullable`
- `lock\_owner text nullable`

BRIDGE / NIGHTHACK BUILD PACKET

- `lock\_expires\_at` timestampz nullable`
- `created\_at` timestampz`
- `updated\_at` timestampz`

impacts

- `id` uuid primary key`
- `run\_id` uuid`
- `file\_path` text`
- `line\_start` int`
- `line\_end` int`
- `snippet` text`
- `reason` text`
- `confidence` numeric`
- `created\_at` timestampz`

migration_plans

- `id` uuid primary key`
- `run\_id` uuid`
- `version` int`
- `title` text`
- `steps` jsonb`
- `patch\_summary` text`
- `risk\_level` text`
- `created\_at` timestampz`

approvals

- `id` uuid primary key`
- `run\_id` uuid`
- `plan\_version` int`
- `participant\_id` uuid`
- `decision` text`
- `note` text nullable`
- `created\_at` timestampz`

comments

- `id` uuid primary key`
- `run\_id` uuid`
- `participant\_id` uuid`
- `body` text`
- `created\_at` timestampz`

run_events

- `id` bigint generated always as identity primary key`
- `run\_id` uuid`
- `sequence` int`
- `actor\_type` text`
- `actor\_id` text nullable`
- `event\_type` text`
- `stage` text`
- `status` text`
- `message` text`

- `metadata.jsonb`
- `created\_at` timestampz`

Event types

- `run.created`
- `change.analysis.started`
- `change.analysis.completed`
- `repo.scan.started`
- `repo.scan.completed`
- `plan.created`
- `plan.approved`
- `patch.started`
- `patch.completed`
- `github.branch.created`
- `github.pr.created`
- `validation.started`
- `validation.updated`
- `validation.passed`
- `validation.failed`
- `comment.created`
- `run.ready\_for\_review`
- `run.failed`

Public API routes

POST `/api/runs/start`

Request:

```
{
  "providerChangeId": "uuid",
  "repositoryId": "uuid"
}
```

Response:

```
{
  "runId": "uuid",
  "status": "queued"
}
```

POST `/api/runs/:runId/advance`

Server/admin-only or protected demo command. Executes exactly one idempotent stage.

Response:

```
{
  "runId": "uuid",
  "previousStatus": "scanning_repo",
  "status": "planning"
}
```

POST `/api/runs/:runId/approve`

Request:

```
{
  "planVersion": 1,
  "decision": "approved",
  "note": "Patch is limited to AtlasPay request fields."
}
```

POST `/api/runs/:runId/comments`

Request:

```
{
  "body": "AtlasPay provider team confirms this field rename."
}
```

GET `/api/runs/:runId`

Returns the room aggregate: run, change, repository, impacts, latest plan, approvals, events, comments, and evidence.

POST `/api/github/webhook`

- Verify signature before parsing.
- Store delivery ID for deduplication.
- Ignore unrelated repos/events.
- Update validation only when commit SHA matches the run.

Realtime channel

Topic: [migration-run:<runId>](#)

Presence payload:

```
{
  "participantId": "uuid",
  "name": "Arush",
  "role": "customer_engineer",
  "avatar": "A",
  "view": "room"
}
```

Broadcast events:

- `run\_status\_changed`
- `comment\_created`
- `approval\_created`
- `evidence\_updated`

The database remains the source of truth. Broadcast payloads tell clients to update immediately; clients refetch when they detect a sequence gap.

State transition guard

A transition is allowed only when:

- the current state matches the expected source state,
- required artifacts for the destination state exist,
- the run lock belongs to the caller,
- the event sequence is incremented atomically.

Example: [validating](#) -> [ready_for_review](#) requires [commit_sha](#), [pull_request_url](#), [validation_url](#), and [validation_conclusion = success](#) for the same SHA.

Demo Fixture Specification

Why a controlled fixture

A six-hour hackathon should not depend on an unpredictable real provider release. Use a realistic fictional provider contract so the migration engine, customer code, and CI behavior are completely controlled while still representing the real problem.

Provider: AtlasPay

v1 request

```
{
  "amount": 2500,
  "currency": "usd",
  "payment_method": "pm_demo_123"
}
```

v2 request

```
{
  "amount": 2500,
  "currency": "usd",
  "payment_method_id": "pm_demo_123"
}
```

Breaking change

For [POST /payments](#):

- Remove request property `payment\_method`.
- Add required request property `payment\_method\_id`.
- Keep data type as string.
- Migration recipe: rename the object key.

Hosted provider assets

- `fixtures/atlaspay-v1.openapi.yaml`
- `fixtures/atlaspay-v2.openapi.yaml`
- `fixtures/atlaspay-v2-migration.md`

Host these on a public production URL or in the demo GitHub repository. The UI should show both source links.

Customer repository

Repository name: [atlas-store-demo](#)

Suggested structure:

```
src/
  atlaspay/client.ts
  checkout/create-payment.ts
  subscriptions/renew.ts
  refunds/retry-charge.ts
tests/
  atlaspay-contract.test.ts
.github/workflows/ci.yml
```

package.json
README.md

Expected impacted files:

- 55. `src/checkout/create-payment.ts`
- 56. `src/subscriptions/renew.ts`
- 57. `src/refunds/retry-charge.ts`

Do not place the deprecated field elsewhere unless it is intentionally used to test false-positive protection.

False-positive guard fixtures

Add these non-target strings:

```
const documentation = "AtlasPay previously called this payment_method";  
const payment_method_label = "Card";  
logger.info({ event: "payment_method_selected" });
```

The patch must not alter them.

Validation design

Create a tiny TypeScript contract package or local type definition representing AtlasPay v2. The customer repository compiles against v2 types. Before patching, TypeScript reports that `payment_method` is invalid and `payment_method_id` is required. After patching, `npm test` or `npm run check` succeeds.

Recommended CI steps:

- npm ci
- npm run typecheck
- npm test

Keep the suite under 20 seconds when a runner starts.

PR body template

Bridge migration: AtlasPay v1 -> v2

AtlasPay changed the `POST /payments` request contract:

- removed `payment\_method`
- added required `payment\_method\_id`

Impacted files

- src/checkout/create-payment.ts
- src/subscriptions/renew.ts
- src/refunds/retry-charge.ts

Patch

Renamed the AtlasPay request property in three verified call sites. No unrelated strings or identifiers were changed.

Validation

- Typecheck: passed
- Tests: passed
- Commit: `<sha>`

Review

Migration room: `<bridge-room-url>`

This is a draft PR and requires human review before merge.

Reset strategy

Maintain a pristine `demo-base` branch. The reset script or manual run should:

58. close or ignore the previous demo PR,
59. delete the previous ``bridge/...`` branch when safe,
60. create a new run ID,
61. point the next migration at ``demo-base``,
62. clear prior comments/approvals for the new room.

Never discover reset behavior for the first time in front of judges.

UI/UX and Design System

Product character

Bridge should feel like a precise developer control plane, not a generic AI dashboard. Use calm dark surfaces, crisp type, restrained electric-blue accents, and evidence-first hierarchy. Avoid purple gradients, glowing robot imagery, excessive terminal chrome, and fake metrics.

Brand direction

- **Name:** Bridge.
- **Descriptor:** API migration control plane.
- **Primary accent:** electric blue.
- **Supporting success:** green used only for verified passing states.
- **Warning:** amber.
- **Failure:** red.
- **Background:** near-black navy rather than pure black.
- **Typography:** geometric sans for headings; highly readable sans for UI; monospaced type only for code and IDs.

Logo concept

A nonliteral mark made from two offset code brackets or vertical rails connected by a single stepped path. The path represents a contract change moving safely from provider to customer. It should also resemble a compact diff or migration graph at favicon size. Do not use a literal suspension bridge.

Main screen: migration room

Header

- Bridge wordmark.
- Provider/version pill: `AtlasPay v1 -> v2`.
- Repository: `acme/atlas-store-demo`.
- Status pill.
- Participant avatar stack.

Left rail: timeline

A vertical stage sequence:

63. Change detected.
64. Repository scanned.
65. Three impacts found.
66. Migration plan approved.
67. Patch committed.
68. CI passed.
69. Ready for review.

Each item includes timestamp, actor, state, and optional evidence link.

Center: impact and patch

Tabs:

- `Impact` - files and code snippets.
- `Plan` - exact deterministic steps.

- `Diff` - compact before/after.
- `Discussion` - comments.

The default tab during the demo should be [Impact](#) until the PR exists, then transition to [Diff](#).

Right rail: evidence

- Draft PR card.
- Commit SHA.
- GitHub Actions/check card.
- Approval card.
- Provider source links.

This panel answers: “How do I know this actually happened?”

Change intake screen

Above the fold:

- Heading: `AtlasPay v2 contains 1 breaking request change.`
- Before/after field visualization.
- Target repository selector fixed to the demo repo.
- Primary button: `Create migration`.

Do not build broad onboarding. The demo should start close to value.

Status language

Use verbs and evidence:

- `Analyzing provider contract`
- `Scanning 18 TypeScript files`
- `Found 3 verified call sites`
- `Created draft pull request #12`
- `GitHub Actions passed for 3f91c2a`

Avoid vague AI phrases like [Thinking...](#) or [Working magic....](#)

Component states

Every critical component needs:

- loading skeleton,
- active state,
- completed state,
- failed state with retry,
- evidence link when complete.

Motion

Use motion to explain state transitions, not decorate the interface:

- timeline item activates,
- impact count increments after scan,
- PR card slides into evidence rail after external creation,
- status changes to green only after validation is verified.

Animations should complete in under 350 ms and never delay interaction.

Responsive behavior

Desktop-first for the demo. At widths below 1100 px:

- collapse right evidence rail into a drawer,
- keep timeline and central content visible,
- preserve participant presence.

Accessibility

- Minimum 4.5:1 text contrast.
- Do not communicate state by color alone.
- Keyboard-focus styles on all actions.
- Code snippets scroll horizontally rather than clipping.
- Status changes should be announced with an accessible live region.

Landing Page Copy and Design Specification

Objective

Make Bridge understandable in ten seconds and credible in thirty. The landing page is not the hackathon demo, but it creates confidence when judges or investors open the public URL after the event.

Navigation

Left: Bridge logo.

Right: [Product](#), [How it works](#), [Security](#), [GitHub](#), and primary CTA [Run a migration](#).

For NightHack, nonfunctional navigation can scroll to sections. Do not build a complex marketing CMS.

Hero

Eyebrow: API migration control plane

Headline:

Turn breaking API changes into tested pull requests.

Supporting copy:

Bridge detects external API changes, maps them to the exact code that will break, generates a bounded migration, validates it in CI, and opens a draft PR your team can review together.

Primary CTA: [See the live migration](#)

Secondary CTA: [View the GitHub PR](#)

Hero visual

A wide product window showing the migration room:

- AtlasPay v1 -> v2 change.
- Three impacted files.
- timeline progressing from detected to CI passed.
- real draft PR card.
- two participant avatars.

Use a subtle background grid that brightens near the cursor only if it is fast to implement. The product UI should remain the focus.

Proof strip

Four concise stages:

- Detect contract changes.
- Find impacted code.
- Generate bounded patches.
- Prove them in CI.

Problem section

Heading: Changelogs announce problems. They do not finish migrations.

Copy:

A provider change can touch dozens of repositories, owners, and release schedules. Today, teams manually interpret the change, search code, coordinate fixes, and hope tests catch the residue. Bridge turns that fragmented process into one auditable migration run.

How it works

1. Detect

Bridge compares official specs and change sources, then normalizes breaking operations.

2. Map

Bridge scans installed repositories and shows the exact files and call sites affected.

3. Migrate

Bridge applies deterministic recipes first and uses AI only for bounded residue.

4. Validate

Bridge opens a draft PR, runs CI, and attaches proof before asking for review.

Collaboration section

Heading: One room for the provider, platform team, and code owner.

Copy:

Comments, approvals, evidence, and status live beside the migration itself. No more reconstructing a breaking change across changelogs, Slack threads, tickets, and pull requests.

Safety section

Heading: Automation without blind trust.

Points:

- Draft PRs by default.
- Least-privilege GitHub access.
- Exact impacted-file evidence.
- CI tied to the migration commit.
- Human approval before merge.

Final CTA

Headline: Your APIs will change. Your code should keep up.

CTA: [Open the AtlasPay migration room](#)

Visual specification

- Near-black navy background.

BRIDGE / NIGHTHACK BUILD PACKET

- White and cool-gray type.
- Electric-blue primary accent.
- Thin blue-gray borders.
- Large editorial headline, restrained copy width.
- Product screenshot dominates the hero.
- No stock illustrations.
- No floating AI orb.
- No purple.
- No fake customer logos or fake usage metrics.

Metadata

Title: [Bridge - Turn API changes into tested pull requests](#)

Description: [Bridge detects breaking API changes, finds impacted code, generates bounded migrations, validates them in CI, and opens draft pull requests.](#)

Demo Script and Runbook

Target length

Four to six minutes. No slides. The product is the presentation.

Presenter setup

Open these tabs before the demo:

- 70. Bridge change intake.
- 71. Bridge migration room in browser A.
- 72. Same room in browser B or incognito for presence.
- 73. GitHub demo repository.
- 74. Backup completed PR and CI run.
- 75. Deployment logs hidden but available to the engineer.

Use 110-125% browser zoom and close unrelated tabs.

Exact demo narrative

0:00-0:35 - Problem and category

Every time an external API changes, engineering teams have to read the changelog, find the affected code, write the migration, test it, and coordinate the rollout. Bridge turns that entire event into a tested draft pull request.

Show the AtlasPay v1 -> v2 change intake.

0:35-1:10 - Detect the breaking change

AtlasPay removed `payment_method` and now requires `payment_method_id` on payment creation. Bridge reads the provider contract, marks this as breaking, and creates a migration against the customer repository.

Click [Create migration](#).

1:10-2:00 - Map customer impact

Open the room as the timeline advances.

Bridge does not blindly ask an agent to rewrite the repo. It maps the provider change to verified call sites first. Here it found exactly three TypeScript files and ignored unrelated strings with similar names.

Open one impacted snippet.

2:00-2:50 - Plan and collaboration

Bring browser B into view so presence is visible.

The provider and customer work in the same migration room. The plan is explicit: rename the AtlasPay request property in these three call sites, change nothing else, then validate the exact commit.

Post the provider comment in browser B and approve the plan.

2:50-3:50 - Execute and open the PR

Bridge creates a migration branch, applies the bounded patch, and opens a draft pull request. It never merges automatically.

Reveal the PR card and click into the real GitHub diff. Scroll only enough to show the three intended changes.

3:50-4:40 - Verify

A patch is not complete because an AI says it is. Bridge ties validation to the migration commit and waits for GitHub Actions.

Return to the room. Show the check URL, commit SHA, and passing status.

4:40-5:15 - Close

Changelogs tell you an API changed. Coding agents can edit code. Bridge owns the missing system between them: detecting the change, proving where it matters, executing the migration, validating it, and coordinating review. It is Dependabot for APIs, with the migration actually done.

End on the green [Ready for review](#) state.

Live-demo controls

- One teammate presents and clicks.
- One teammate controls browser B and provider comment.
- One teammate watches logs and CI status.
- Do not switch drivers mid-demo.

Reset procedure

76. Create a new run from `demo-base`.
77. Confirm no active branch uses the new run ID.
78. Confirm both browser identities join the room.
79. Keep the completed backup run open.
80. Run the exact demo once after reset.

Failure recoveries

Actions queue is slow

Say:

The workflow is running against this exact commit. To keep the demo moving, here is the completed run from our previous migration using the same deployed path.

Open the backup completed run. Do not pretend the queued run passed.

GitHub write fails

Show the generated patch preview and completed backup PR, then state that the live GitHub request failed. This is weaker but honest.

Realtime fails

Use two windows with the persisted comments already visible after refresh. Continue the core migration story.

Spec diff fails

Use the normalized change record already stored in the app and explain that the official source links are attached.

Demo anti-patterns

- Do not explain database tables.
- Do not show code unless asked.
- Do not wait silently for CI.
- Do not claim support for every language.
- Do not say the AI is autonomous.
- Do not introduce unrelated future features before the core story lands.

Pitch and Judge Q&A

20-second pitch

Bridge turns breaking external API changes into tested draft pull requests. It detects the provider change, finds the exact customer code that will break, generates a bounded migration, validates the commit in CI, and gives the provider and customer one shared room to review it.

60-second pitch

APIs are now core infrastructure, but migrations are still coordinated through changelogs, search, tickets, Slack threads, and manual pull requests. Bridge is the missing migration control plane. When an API provider changes a contract, Bridge maps that change into each installed customer repository, shows the exact impacted call sites, applies deterministic migration recipes first, validates the exact commit, and opens a draft PR with a collaborative migration room. We start with GitHub and TypeScript integrations where the change can be bounded and proven. Over time, Bridge becomes the system providers use to safely move their entire customer ecosystem forward.

Why now

- API surface area and external integration dependence continue to expand.
- Providers increasingly publish machine-readable contracts.
- GitHub provides a standard distribution and pull-request workflow.
- Modern code models make long-tail migration assistance possible, but trust requires deterministic rules, CI, and human review.

Differentiation

Versus Dependabot/Renovate

They update package versions. Bridge understands provider contract changes and maps them to integration-specific code migrations.

Versus API diff tools

They identify contract differences. Bridge carries the difference through customer impact, patching, CI, PR creation, and coordination.

Versus AI coding agents

They can modify code after a person gives them a task. Bridge detects the task from the provider change, scopes it, proves affected sites, and manages the migration lifecycle.

Versus SDK generators

They regenerate clients. Bridge handles application code and workflows around a provider integration.

Defensibility

The long-term moat is not the first codemod. It is the migration graph:

- provider changes,
- customer code patterns,
- verified impacts,
- successful recipes,
- validation outcomes,
- time-to-migrate,
- failure and rollback evidence.

This dataset can improve confidence, provider-specific recipes, prioritization, and enterprise policy. Distribution through provider ecosystems and GitHub installations creates additional defensibility.

Business model

- Starter: org/repo subscription.
- Growth: higher repo limits, policy controls, private runners, and audit features.
- Enterprise/provider: annual contract for ecosystem migration visibility and customer coordination.
- Usage add-ons: isolated validation compute or high migration volume.

Avoid token-based pricing as the primary model; buyers pay for protected integrations and avoided migration work.

Judge questions

“Is this just a code agent with a trigger?”

No. A code agent starts after a human frames a task. Bridge owns the upstream and downstream system: provider change detection, repo-specific impact mapping, migration policy, deterministic recipes, validation tied to the exact commit, PR evidence, and cross-company coordination.

“Why would API providers pay?”

Faster customer migrations reduce support load, version fragmentation, and delayed deprecation. The initial wedge can sell to customer engineering teams, then use migration evidence to pull providers into an ecosystem console.

“How do you prevent broken AI patches?”

AI is not trusted by default. Bridge uses deterministic transformations where possible, limits changes to verified impacts, opens draft PRs, validates the exact commit, and requires human review before merge.

“Why the multiplayer room?”

API migrations cross organizational boundaries. The provider understands the contract; the customer understands the application. Today evidence and decisions fragment across channels. The room makes the migration itself the shared source of truth.

“Can this support arbitrary APIs?”

Not initially. The credible wedge is one provider ecosystem, GitHub, and one language family. Generalization comes after enough successful, verified migrations.

“What stops GitHub or an AI coding company from building it?”

They can build pieces. Bridge’s advantage must come from provider distribution, deep migration workflows, provider-specific recipe data, cross-repository impact visibility, and a reputation for safe execution. A generic agent platform has less incentive to own provider/customer coordination.

“What is the biggest risk?”

Low-confidence patches. The company fails if it broadens faster than it can prove correctness. The product must earn trust in narrow ecosystems before expanding.

“What would you build next?”

Real GitHub App onboarding, one real provider design partner, repository monitors, policy controls, and a recipe engine that combines OpenAPI diffs, structural search, deterministic transforms, and model-assisted residue.

QA Test Plan

Release philosophy

Test the demo path, not the theoretical platform. A polished dashboard with one broken external write is a failed demo.

P0 test matrix

Deployment

- Public URL loads on two unrelated devices.
- Refreshing the room preserves state.
- Production environment has all required variables.
- No console crash on a clean browser session.

Change analysis

- v1/v2 sources load from production.
- Normalized diff contains the correct removed and required fields.
- Re-running analysis is idempotent.
- Invalid spec produces a clear error and retry.

Impact scan

- Finds exactly three intended files.
- Does not modify false-positive strings.
- Stores line numbers and snippets.
- Handles missing repository file cleanly.

Patch

- Changes only `payment\_method` object keys in scoped call sites.
- Produces parseable TypeScript.
- Patch preview matches committed diff.
- Re-running does not duplicate or corrupt changes.

GitHub

- Installation/token path works in production.
- Branch name is unique.
- Commit exists on expected branch.
- Draft PR targets expected base branch.
- PR body includes room and evidence links.

Validation

- Status is tied to the migration commit SHA.
- Passing check sets `ready\_for\_review`.
- Failed check sets `validation\_failed` and shows logs link.
- Stale successful check from another SHA is rejected.

Realtime

- Browser A sees browser B presence.
- Comment appears in both sessions.
- Approval appears in both sessions.

- Reconnect after temporary network loss restores state.

Security

- GitHub credentials do not appear in browser source, network payloads, or logs.
- Webhook signature validation rejects invalid requests.
- Repo identifiers are server-validated.
- User-supplied comments are escaped.

Demo rehearsal tests

Run these in order:

81. Cold browser, new run, complete flow.
82. Second new run after reset.
83. CI slow-path recovery.
84. GitHub write failure fallback.
85. Realtime disconnect fallback.
86. Presenter-only rehearsal with no engineer intervention.

Visual QA

- No clipped code snippets.
- Status colors have text labels.
- Timeline timestamps are readable.
- Buttons do not move as state changes.
- PR and CI links are obvious.
- At 110-125% browser zoom, the critical room fits without chaotic scrolling.

Bug severity

P0

Demo cannot complete, production unavailable, wrong code changed, fake/incorrect evidence, credentials exposed.

P1

Major surface unusable, room state does not update, error not recoverable, participant cannot approve.

P2

Visual inconsistency, minor copy issue, slow noncritical animation.

After integration freeze, fix only P0 and high-confidence P1 issues.

Definition of done

- Two clean end-to-end rehearsals.
- One teammate who did not build the feature can operate the demo.
- Backup run and PR are accessible.
- All P0 tests pass.
- No known secret exposure.

Deployment, Security, and Environment Checklist

Production topology

- Web/server deployment.
- Supabase project.
- GitHub App and webhook endpoint.
- GitHub demo organization/repository.
- Model API only if used for optional explanation.
- Public provider-spec URLs.

Environment variables

Suggested names:

```

NEXT_PUBLIC_APP_URL
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
SUPABASE_SECRET_KEY
GITHUB_APP_ID
GITHUB_APP_PRIVATE_KEY
GITHUB_WEBHOOK_SECRET
GITHUB_DEMO_INSTALLATION_ID
GITHUB_DEMO_OWNER
GITHUB_DEMO_REPO
MODEL_API_KEY
DEMO_MODE

```

Never commit real values. Normalize multiline private keys on the server. Use separate development and production secrets.

GitHub security

- Install the app only on the demo repository.
- Request least privilege.
- Use short-lived installation tokens.
- Verify webhook signatures.
- Store and deduplicate webhook delivery IDs.
- Never send repository credentials to the browser.
- Do not allow arbitrary owner/repo input to reach write operations.
- Ensure branch creation and PR creation are scoped to the configured demo repository.

Supabase security

- Persist room data behind Row Level Security where practical.
- Use server-side service credentials only on trusted routes.
- Keep publishable key in the browser; keep secret/service key server-side.
- Do not put secrets in Realtime presence payloads.
- Persist comments/approvals; treat presence as ephemeral.

Deployment checklist

- Production build succeeds locally or in CI before deployment.
- Correct Node/runtime version configured.

- Database migrations applied to production.
- Webhook URL points to production.
- GitHub App callback and setup URLs are correct.
- Public app URL matches the room links inserted into PR bodies.
- Health route returns success.
- Logs can be filtered by migration run ID.
- Error reporting is enabled or logs are easy to inspect.

Observability

For each stage log:

- run ID,
- stage,
- attempt,
- duration,
- result,
- external request category,
- external ID after successful creation.

Never log:

- tokens,
- private keys,
- full private repository files,
- unredacted webhook signatures.

Demo mode

DEMO_MODE may:

- restrict product access to configured fixtures,
- expose a reset control to team members,
- use named demo identities,
- shorten polling intervals,
- disable unrelated routes.

It must not fabricate external success. A demo mode should reduce scope, not falsify evidence.

Final credential audit

- Search repository history for key prefixes.
- Rotate any token ever pasted into chat, issue text, or client code.
- Remove temporary personal access tokens after the event.
- Keep GitHub App permissions narrow after the demo.

Risk Register and Fallbacks

Risk hierarchy

1. GitHub Actions latency

Probability: high.

Impact: high.

Mitigation: keep test suite tiny, trigger validation early, maintain a completed backup run from the same deployed flow, and never wait silently.

2. GitHub authentication/permissions failure

Probability: medium.

Impact: critical.

Mitigation: verify installation access immediately, create a backend permission diagnostic, and keep a narrow server-side fallback credential only if event/security rules allow it.

3. Patch changes wrong text

Probability: medium.

Impact: critical.

Mitigation: AST transform or exact guarded matches, false-positive fixtures, diff assertion, and no auto-merge.

4. Realtime instability

Probability: medium.

Impact: medium.

Mitigation: persist all important collaboration data; presence is an enhancement. Refetch on reconnect.

5. Deployment outage or broken latest build

Probability: medium.

Impact: critical.

Mitigation: pin the last known-good deployment after integration freeze and stop speculative commits.

6. Overbuilding

Probability: very high.

Impact: critical.

Mitigation: P0 board, 25-minute task rule, explicit cut order, and no generalized provider/language support.

7. Demo confusion

Probability: medium.

Impact: high.

Mitigation: one story, one provider, one repo, one change, one presenter, no architecture monologue.

Fallback levels

Level A - Fully live

- Live change analysis.

- Live scan.
- Live patch and PR.
- Live or already-completed real CI.
- Live presence and approval.

Level B - Real external output, cached orchestration

- Use a persisted completed run created by the deployed product.
- Open the real PR and real CI evidence.
- Demonstrate live comments/presence.

Level C - Patch preview plus verified prior PR

- Show live analysis and impact.
- Show generated patch preview.
- Open a previous real PR/check from the same fixture.
- State exactly which write failed.

Level D - Screen recording

Only when the production path is unavailable. Play a concise recording of the same deployed product and then answer questions using the live codebase. This is the weakest option and must not be presented as live.

Kill switches

- Disable optional AI explanation if it slows or destabilizes the run.
- Disable comments before disabling presence.
- Disable custom check-run creation and rely on GitHub Actions status if permissions become complex.
- Use a preconfigured repository instead of live installation onboarding.
- Use one combined room page instead of separate provider/customer dashboards.

Honest demo language

Never say “Bridge completed this live” when showing a prior run. Say “This is the completed run from the same deployed workflow.” Credibility is more valuable than pretending a network queue behaved perfectly.

Team Roles and War Room

Three-person team

Person A - Orchestrator/GitHub owner

Owns:

- GitHub App/authentication.
- Repository adapter.
- Branch, commit, PR, validation polling.
- External integration debugging.

Person B - Product/realtime owner

Owns:

- Next.js pages and migration room.
- Supabase schema and realtime.
- Timeline, evidence panel, comments, approval.
- Production deployment.

Person C - Change/patch/demo owner

Owns:

- OpenAPI fixture and normalized diff.
- Impact scanner and patch engine.
- Demo repository and tests.
- Demo script, QA, fallback assets.

At integration freeze, Person C becomes presenter, Person B operates second browser, and Person A watches logs.

Two-person team

Person A - Backend/integrations

- GitHub App.
- change detector.
- scanner/patcher.
- PR and CI.

Person B - Product/deployment

- data model.
- room UI.
- realtime.
- deployment.
- demo and QA.

Both integrate at the top of every hour. Do not wait until the last 90 minutes to combine branches.

Branch strategy

- `main` must remain deployable.
- Short-lived branches by feature.
- Merge small vertical increments.

- No long-running rewrite branch.
- Tag or record the last known-good deployment at integration freeze.

Communication protocol

Every 25-30 minutes, each person reports:

87. what is now demo-visible,
88. what is blocked,
89. what they will finish next,
90. whether scope must be cut.

No status speech longer than one minute.

Shared board

Columns:

- `P0 Next`
- `Building`
- `Needs integration`
- `Demo verified`
- `Cut`

Each card must have one owner and one observable acceptance condition.

Handoff contract

A feature is not handed off with “the code is done.” It is handed off with:

- route or function signature,
- expected payload,
- example data,
- error behavior,
- deployed or testable state,
- known limitations.

War-room discipline

- Use one shared text channel for blockers.
- Put credentials only in the proper secret manager, never chat.
- One person controls production deploys after integration freeze.
- Do not refactor working code during the final hour.
- The presenter stops coding 30-45 minutes before judging and rehearses.

Post-Hackathon Roadmap

First 72 hours

- Preserve the winning demo deployment.
- Write a public technical walkthrough and short demo video.
- Contact 15-20 engineers who own Stripe, Shopify, Slack, Twilio, or GitHub integrations.
- Ask for the last API migration that consumed meaningful engineering time.
- Recruit three design partners around one ecosystem.
- Instrument migration completion, false positives, manual edits, and time saved.

First 30 days

Product

- Real GitHub App installation flow.
- Repository selection and monitoring.
- One real provider ecosystem.
- Robust spec/changelog ingestion.
- Deterministic recipe registry.
- Better TypeScript structural analysis.
- Policy controls and audit log.
- Private execution model for customer code.

Validation goals

- Ten real migration histories collected.
- Three design partners connect repos.
- At least one migration PR is merged.
- Customer reports measurable time saved.
- False-positive rate is understood, not hand-waved.

Days 31-90

- Add provider-specific migration packs.
- Handle multi-step changes, not only field renames.
- Add a review queue across repositories.
- Add model-assisted residue behind confidence thresholds.
- Add self-hosted or customer-controlled execution path.
- Pilot provider console with one API company.

Metrics

Product quality

- impact precision,
- impact recall on known migrations,
- patch acceptance rate,
- CI pass rate,
- median manual edits after Bridge,
- time from provider change to draft PR,
- time from PR to merge.

Business

- installed organizations,
- monitored integrations,
- migration events per org,
- active design partners,
- willingness to pay,
- expansion from customer-side to provider-side.

Strategic sequencing

91. Win one provider-language pair.
92. Prove safe, merged migrations.
93. Build repeatable recipe and evidence data.
94. Expand to adjacent providers or languages.
95. Sell ecosystem visibility and coordination to providers.

Do not expand simply because a demo works. Expand when accuracy and adoption justify it.

Claude Code Master Prompt

Copy the text below into Claude Code at the root of a clean repository together with this packet.

You are the lead engineer for Bridge, a production-deployed NightHack project.

Bridge turns a third-party API breaking change into a repo-specific, tested draft pull request and a shared migration room. The NightHack build window is six hours. The demo must work from a public URL and show one controlled AtlasPay v1 -> v2 migration against one TypeScript GitHub repository.

READ FIRST

- 00_READ_ME_FIRST.md
- 02_NIGHTHACK_EXECUTION_PLAN.md
- 03_MVP_PRD.md
- 04_TECHNICAL_ARCHITECTURE.md
- 05_DATA_MODEL_AND_API_CONTRACTS.md
- 06_DEMO_FIXTURE_SPEC.md
- 11_QA_TEST_PLAN.md
- 17_FINAL_SUBMISSION_CHECKLIST.md

NON-NEGOTIABLES

1. Build one vertical slice only.
2. Keep main deployable.
3. Use deterministic migration logic for the AtlasPay field rename.
4. Do not put the LLM on the critical path.
5. Never expose GitHub or Supabase secret credentials to the client.
6. Never claim a GitHub action succeeded without verifying the external result.
7. Open draft PRs only. Never merge.
8. Persist all critical state; presence may be ephemeral.
9. Every stage must be idempotent and create an audit event.
10. Stop adding features at integration freeze.

FIRST ACTIONS

1. Inspect the repository and available environment.
2. Create IMPLEMENTATION_PLAN.md with P0 tasks mapped to the six-hour schedule.
3. Create a minimal production deploy immediately.
4. Add database migrations for the packet's core tables.
5. Implement one end-to-end thin slice using seeded data before replacing stages with real integrations.

IMPLEMENTATION ORDER

- A. Production deployment and room route.
- B. Persisted run and timeline.
- C. AtlasPay spec diff normalization.
- D. GitHub repository file fetch and exact impact scan.
- E. Deterministic patch preview.
- F. Branch, commit, and draft PR.
- G. GitHub Actions status for exact commit SHA.
- H. Supabase presence, comments, and approval.
- I. Error states, reset flow, and demo polish.

INTERFACES

Create explicit adapters for:

- ChangeDetector
- RepositoryClient

BRIDGE / NIGHTHACK BUILD PACKET

- ImpactScanner
- PatchEngine
- ValidationClient
- RealtimePublisher

Keep AtlasPay-specific rules inside a migration recipe module, not UI components.

QUALITY GATES

- Typecheck and tests after each meaningful change.
- Confirm production deployment after each integration milestone.
- Add a false-positive test for strings containing payment_method.
- Assert that committed diff matches patch preview.
- Validate GitHub check/workflow SHA before success.
- Run the complete demo twice from reset.

REPORT AFTER EACH PHASE

- files changed,
- behavior now working,
- tests run,
- production URL status,
- known risk,
- next P0 task.

DO NOT

- generalize to arbitrary APIs,
- build billing or enterprise onboarding,
- create a generic agent chat,
- add multiple dashboards,
- refactor working code during the final hour,
- hide failures with fake success states.

Begin by creating the implementation plan and the smallest deployed room with seeded data. Then execute the P0 path in order.

Final Submission Checklist

Product

- ☐ Public production URL loads.
- ☐ Change intake shows AtlasPay v1 -> v2.
- ☐ New migration run can be created.
- ☐ Exactly three impacted files appear.
- ☐ Patch changes only intended object keys.
- ☐ Draft PR exists in the real repository.
- ☐ PR body links to the production room.
- ☐ CI/check result is real and tied to the migration SHA.
- ☐ Room reaches `Ready for review` only after success.
- ☐ Two sessions show live presence.
- ☐ Comment replicates.
- ☐ Approval replicates and persists.

Deployment and security

- ☐ Latest known-good deployment pinned.
- ☐ Database migrations applied.
- ☐ Webhook points to production.
- ☐ No secrets in browser source or repository.
- ☐ GitHub App limited to demo repo.
- ☐ Logs are accessible and filtered by run ID.
- ☐ Reset flow works.

Demo assets

- ☐ Fresh live run prepared.
- ☐ Completed backup run prepared.
- ☐ Completed backup PR and CI tab prepared.
- ☐ Short screen recording saved locally and in cloud.
- ☐ Screenshots of each key state saved.
- ☐ Browser A and B identities verified.
- ☐ Presenter script available offline.

Demo environment

- ☐ Laptop charged.
- ☐ Charger and adapter packed.
- ☐ Hotspot tested.
- ☐ Notifications disabled.
- ☐ Browser zoom set.

BRIDGE / NIGHTHACK BUILD PACKET

- ☐ Unrelated tabs closed.
- ☐ GitHub session active.
- ☐ No password manager popups expected.
- ☐ System updates paused.

Rehearsal

- ☐ End-to-end rehearsal #1 passed.
- ☐ End-to-end rehearsal #2 passed.
- ☐ CI slow-path recovery rehearsed.
- ☐ GitHub failure fallback rehearsed.
- ☐ Presenter finishes in under six minutes.
- ☐ Every teammate can state the one-sentence pitch.

Final rule

After the last successful rehearsal, do not add a feature. Only fix a confirmed demo blocker.

Verified Technical References

The architecture choices in this packet were checked against current official documentation. GitHub Apps support repository-scoped permissions and can create pull-request/check workflows; GitHub recommends choosing only the permissions needed. Supabase Realtime Presence supports synchronized participant state in shared channels. Re-check exact API versions and credential types on build day.

- [GitHub Apps overview](#)
- [Choosing GitHub App permissions](#)
- [GitHub Checks API guide](#)
- [GitHub pull request REST API](#)
- [Triggering GitHub Actions workflows](#)
- [Supabase Realtime Presence](#)